

Updated 2203 GMT+8 Nov 3, 2025

2025 fall

说明：

1. 解题与记录：

对于每一个题目，请提供其解题思路（可选），并附上使用Python或C++编写的源代码（确保已在OpenJudge，Codeforces，LeetCode等平台上获得Accepted）。请将这些信息连同显示“Accepted”的截图一起填写到下方的作业模板中。（推荐使用Typora <https://typoraio.cn> 进行编辑，当然你也可以选择Word。）无论题目是否已通过，请标明每个题目大致花费的时间。

2. **提交安排：**提交时，请首先上传PDF格式的文件，并将.md或.doc格式的文件作为附件上传至右侧的“作业评论”区。确保你的Canvas账户有一个清晰可见的本人头像，提交的文件为PDF格式，并且“作业评论”区包含上传的.md或.doc附件。

3. **延迟提交：**如果你预计无法在截止日期前提交作业，请提前告知具体原因。这有助于我们了解情况并可能为你提供适当的延期或其他帮助。

请按照上述指导认真准备和提交作业，以保证顺利完成课程要求。

1. 题目

T51.N皇后

backtracking, <https://leetcode.cn/problems/n-queens/>

思路：

“八皇后”的拓展。首先，和“八皇后”一样，所有皇后不能处于横竖斜位置，此时可以直接用数学判断，即列与所有的皇后均不同，且行数之差等于列数之差的绝对值。其次，解决问题的方法同样是回溯，使用for循环遍历行数递增下的不同列index，直到满足皇后的位置，进入下一行的回溯，否则返回；长度达标时，终止回溯。

代码：

```
from collections import deque
from typing import List, Optional

def if_ok(past_queens: List, index: int):
    for i in range(len(past_queens)):
        if index == past_queens[i] or (len(past_queens)-i) == abs(index -
past_queens[i]):
```

```
        return False
    return True
```

```
class Solution:
```

```
    def solveNQueens(self, n: int) -> List[List[str]]:
        result = []
        track = []
```

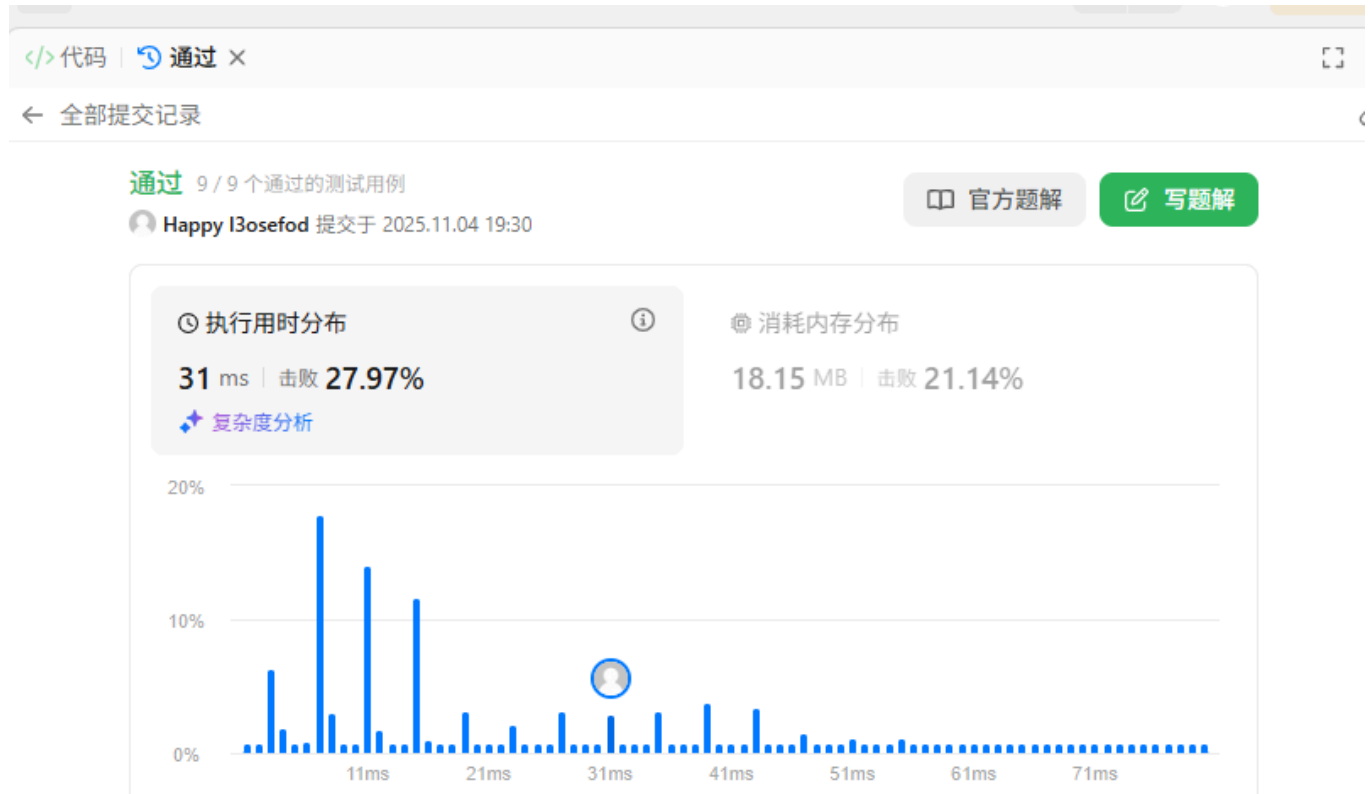
```
    def backtrack():
        for j in range(n):
            if if_ok(track, j):
                track.append(j)
                if len(track) == n:
                    result.append(track[:])
                else:
                    backtrack()
                track.pop()
```

```
    for i in range(n):
        track.append(i)
        if len(track) == n:
            result.append(track[:])
        else:
            backtrack()
        track.pop()
```

```
    result_print = []
    for i in result:
        temp_print = []
        for j in i:
            temp = ['.'] * n
            temp[j] = 'Q'
            temp_print.append(''.join(temp))
        result_print.append(temp_print)
```

```
    return result_print
```

代码运行截图（至少包含有"Accepted"）



M22275: 二叉搜索树的遍历

<http://cs101.openjudge.cn/practice/22275/>

思路：

主要利用了二叉搜索树的性质：比根节点小的节点在根的左子树，比根节点大的节点则在右子树。因此，我们利用前序遍历首先遍历根节点的特点，将列表的第一位视为根节点，遍历剩余的列表，将小于第一位的部分划归左子树，大于的部分划归右子树，再对划归左右子树的列表按上文的方式迭代即可。

代码：

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def build_tree(nums):
    if not nums:
        return None

    root = TreeNode(nums[0])
    for i in range(1, len(nums)+1):
```

```

        if i == len(nums) or nums[i] > root.val:
            root.left = build_tree(nums[1:i])
            root.right = build_tree(nums[i:])
            break
    return root

def postorder(node, trace):
    if node:
        trace = postorder(node.left, trace)
        trace = postorder(node.right, trace)
        trace.append(str(node.val))
    return trace

def main():
    n = int(input())
    nums = [int(x) for x in input().split()]
    root = build_tree(nums)
    result = postorder(root, trace=[])
    print(' '.join(result))

if __name__ == '__main__':
    main()

```

代码运行截图（至少包含有"Accepted"）

#50705424提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: Accepted

源代码

```

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def build_tree(nums):
    if not nums:
        return None

    root = TreeNode(nums[0])
    for i in range(1, len(nums)+1):
        if i == len(nums) or nums[i] > root.val:
            root.left = build_tree(nums[1:i])

```

基本信息

#: 50705424
 题目: 22275
 提交人: 22n2200011816(略约横溪)
 内存: 4060kB
 时间: 23ms
 语言: Python3
 提交时间: 2025-11-04 22:46:10

M25145: 猜二叉树（按层次遍历）

<http://cs101.openjudge.cn/practice/25145/>

思路：

和之前将前序、中序转化为后序类似，这道题目也是已知后序、中序，建立树之后再按层次遍历。同样按照后序遍历先左、再右、后根的遍历顺序，取列表的最后一位作为根，再在中序遍历列表中，取根节点左侧为左子树，右侧为右子树。左、右侧列表再分别按上述顺序取根、建左子树、右子树。对按层遍历，则利用while循环，将每层根节点的左右子节点加入队列，再从队列中popleft——将左右子节点加入队列，就能按照层的循序输出结点。

代码：

```
from collections import deque

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def node_in_post_and_mid(mid, parent_post):
    mid_set = set(mid)
    post = []
    for i in parent_post:
        if i in mid_set:
            post.append(i)
    return post

def build_tree(post: str, mid: str):
    root = TreeNode(post[-1])
    index = 0
    for i in range(len(mid)):
        if mid[i] == post[-1]:
            index = i
            break
    left_mid = mid[:index]
    right_mid = mid[index + 1:]
    if len(left_mid) > 0:
        left_post = node_in_post_and_mid(left_mid, post)
        root.left = build_tree(left_post, left_mid)
    if len(right_mid) > 0:
        right_post = node_in_post_and_mid(right_mid, post)
        root.right = build_tree(right_post, right_mid)

    return root
```

```

def levelorder(node: TreeNode, trace: list):
    if node:
        q = deque([node])
        while q:
            temp_node = q.popleft()
            trace.append(temp_node.val)
            if temp_node.left is not None:
                q.append(temp_node.left)
            if temp_node.right is not None:
                q.append(temp_node.right)
        return trace

def main():
    n = int(input())
    for i in range(n):
        mid = input()
        post = input()
        root = build_tree(post, mid)
        print(''.join(levelorder(root, trace=[])))

if __name__ == '__main__':
    main()

```

代码运行截图（至少包含有"Accepted"）

#50783544提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: **Accepted**

源代码

```

from collections import deque

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def node_in_post_and_mid(mid, parent_post):
    mid_set = set(mid)
    post = []
    for i in parent_post:
        if i in mid_set:
            post.append(i)
    return post

```

基本信息

#: 50783544
 题目: 25145
 提交人: 22n2200011816(略约横溪)
 内存: 3672kB
 时间: 23ms
 语言: Python3
 提交时间: 2025-11-10 19:49:03

T20576: printExp（逆波兰表达式建树）

思路：这题和之前的中序表达式转后序表达式很类似，区别此处将表达式建成树，并判断打印中序表达式时，哪些环节需要加括号。对此，在做题时，就用以上思路：

1. 分析运算符: 在python中，not运算符的优先级最高，其次为and，再次为or。其中，not最为特殊，由于非门只有一个输入端，因此，仅需在受非门影响的结点中加入'have_not'这一属性，打印时随节点打印即可。而对and和or运算符，则按照优先级顺序，先建and联系的两个输入，再建or联系的输入；
2. 建树：利用栈的方式，左括号将index入栈，右括号出栈，随后将中间的文段按1.中的要求处理，迭代建立结点。最终处理整段文字，完成建树。
3. 输出：最后，是对树进行中序遍历，并判断哪些情况需要括号。首先，如果节点的have_not等于1，则立刻输出一个not；其次，判断需要括号的情况：如果子节点是数字，由于数字节点一定是叶节点，不涉及运算符的问题，不需要括号；如果子节点和父节点运算符相同，或子节点是and，父节点是or，根据运算符的运算规则，也不需要括号。其他情况则需要加上括号。将结果输出即可。

代码

```
class TreeNode:
    def __init__(self, val=None, left=None, right=None, have_not=0):
        self.val = val
        self.left = left
        self.right = right
        self.have_not = have_not

def isTreeNode(node_or_s):
    if isinstance(node_or_s, TreeNode):
        return node_or_s
    else:
        return TreeNode(node_or_s)

def middle_to_node(s):
    j = 0
    while j < len(s):
        if s[j] == 'not':
            temp = isTreeNode(s[j+1])
            temp.have_not = 1
            del s[j:j+2]
            s.insert(j, temp)
        j += 1
```

```

sign = ['and', 'or']
for i in range(2):
    j = 0
    while j < len(s):
        if s[j] == sign[i]:
            temp = TreeNode(s[j])
            temp.left = isTreeNode(s[j - 1])
            temp.right = isTreeNode(s[j + 1])
            del s[j-1:j+2]
            s.insert(j-1, temp)
            j -= 1
        j += 1
return s[0]

```

```

def need_brackets(val1, val2):
    temp = ['and', 'or']
    if val1 not in temp:
        return 0
    if val1 == val2:
        return 0
    if val1 == 'and' and val2 == 'or':
        return 0
    return 1

```

```

def midorder(trace, node: TreeNode, parent_node: TreeNode, depth: int):
    if node is None:
        return trace
    depth += 1
    if node.have_not == 1:
        trace.append('not')
    if depth > 1 and need_brackets(node.val, parent_node.val):
        trace.append('(')
    trace = midorder(trace, node.left, node, depth)
    trace.append(node.val)
    trace = midorder(trace, node.right, node, depth)
    if depth > 1 and need_brackets(node.val, parent_node.val):
        trace.append(')')
    return trace

```

```

def main():
    s = input().split()
    stack = []
    j = 0

```

```

while 1:
    if s[j] == '(':
        stack.append(j)
    elif s[j] == ')':
        index = stack.pop()
        temp = middle_to_node(s[index + 1:j])
        del s[index:j + 1]
        s.insert(index, temp)
        j -= (j - index + 1)
    j += 1
    if j == len(s):
        break
s = middle_to_node(s)
trace = []
print(' '.join(midorder(trace, s, parent_node=TreeNode(), depth=0)))
return 0

if __name__ == '__main__':
    main()

```

代码运行截图（至少包含有"Accepted"）

#50734525提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: **Accepted**

源代码

```

class TreeNode:
    def __init__(self, val=None, left=None, right=None, have_not=0):
        self.val = val
        self.left = left
        self.right = right
        self.have_not = have_not

def isTreeNode(node_or_s):
    if isinstance(node_or_s, TreeNode):
        return node_or_s
    else:
        return TreeNode(node_or_s)

def middle_to_node(s):
    j = 0
    while j < len(s):
        if s[j] == 'not':
            temp = isTreeNode(s[j+1])
            temp.have_not = 1

```

基本信息

#: 50734525
 题目: 20576
 提交人: 22n2200011816(略约横溪)
 内存: 4140kB
 时间: 20ms
 语言: Python3
 提交时间: 2025-11-06 22:57:12

T04080:Huffman编码树

greedy, <http://cs101.openjudge.cn/practice/04080/>

思路：在讲义和网络上了解了huffman编码的原理之后就很简单了。这里推荐一下我找到的网页[哈夫曼编码HuffmanCoding原理详解 - 糖豆爸爸 - 博客园](#)，感觉比讲义里面清楚很多。在学会之

后，根据huffman编码的原理，按照最小堆来管理权重，将最小权重的两个节点pop出堆，weight相加后，形成新节点push入堆即可。对带权外部路径长度，则将前序遍历简单修改，记录深度的同时，如果是叶节点，返回weight*depth，否则对子节点进行回溯即可。

代码

```
import heapq

class Node:
    def __init__(self, weight, char=None):
        self.weight = weight
        self.char = char
        self.left = None
        self.right = None

    def __lt__(self, other):
        if self.weight == other.weight:
            return 0
        return self.weight < other.weight

def min_length(node: Node, depth: int):
    result = 0
    if node.left is None and node.right is None:
        return node.weight * depth
    if node.left is not None:
        result += min_length(node.left, depth + 1)
    if node.right is not None:
        result += min_length(node.right, depth + 1)
    return result

def main():
    n = int(input())
    nums = [Node(int(x)) for x in input().split()]
    heapq.heapify(nums)

    while len(nums) > 1:
        left = heapq.heappop(nums)
        right = heapq.heappop(nums)
        temp_node = Node(left.weight + right.weight)
        temp_node.left = left
        temp_node.right = right
        heapq.heappush(nums, temp_node)

    print(min_length(nums[0], depth=0))
```

```
if __name__ == '__main__':  
    main()
```

代码运行截图（至少包含有"Accepted"）

#50793748提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: Accepted

源代码

```
import heapq  
  
class Node:  
    def __init__(self, weight, char=None):  
        self.weight = weight  
        self.char = char  
        self.left = None  
        self.right = None  
  
    def __lt__(self, other):  
        if self.weight == other.weight:  
            return 0  
        return self.weight < other.weight  
  
def min_length(node: Node, depth: int):  
    result = 0  
    if node.left is None and node.right is None:  
        return node.weight * depth  
    if node.left is not None:  
        result += min_length(node.left, depth + 1)  
    if node.right is not None:  
        result += min_length(node.right, depth + 1)  
    return result  
  
def main():  
    n = int(input())  
    nums = [Node(int(x)) for x in input().split()]  
    heapq.heapify(nums)
```

基本信息

#: 50793748
题目: 04080
提交人: 22n2200011816(略约横溪)
内存: 3684kB
时间: 22ms
语言: Python3
提交时间: 2025-11-11 16:21:16

M04078: 实现堆结构

<http://cs101.openjudge.cn/practice/04078/>

要求手搓堆实现。

思路：在学习讲义之后，利用完全二叉树的性质，以一个列表来形成堆。所用的思路和讲义很类似，利用完全二叉树父节点在 $(index - 1) // 2$ ，子节点在 $2 * index + 1$ 和 $2 * index + 2$ 的性质，利用好while循环对列表进行交换，并利用好边界条件即可完成堆的insert和delete。

代码：

```
class BinaryHeap:  
    def __init__(self):  
        self.heap = []  
  
    def insert(self, n):  
        self.heap.append(n)  
        temp_index = len(self.heap) - 1
```

```

        while (temp_index - 1)//2 >= 0:
            parent_index = (temp_index - 1)//2
            if self.heap[parent_index] > self.heap[temp_index]:
                self.heap[parent_index], self.heap[temp_index] =
self.heap[temp_index], self.heap[parent_index]
                temp_index = parent_index
            else:
                break

    def delete(self):
        self.heap[0], self.heap[-1] = self.heap[-1], self.heap[0]
        q = self.heap.pop()
        if len(self.heap) > 1:
            temp_index_0 = 0
            while 2*temp_index_0 + 1 < len(self.heap):
                child_index = 2 * temp_index_0 + 1
                if 2 * temp_index_0 + 2 < len(self.heap):
                    if self.heap[2 * temp_index_0 + 1] > self.heap[2 *
temp_index_0 + 2]:
                        child_index = 2 * temp_index_0 + 2

                if self.heap[temp_index_0] > self.heap[child_index]:
                    self.heap[temp_index_0], self.heap[child_index] =
self.heap[child_index], self.heap[temp_index_0]
                    temp_index_0 = child_index
                else:
                    break

        return q

def main():
    n = int(input())
    heap = BinaryHeap()
    for i in range(n):
        os = [int(x) for x in input().split()]
        if os[0] == 1:
            heap.insert(os[1])
        else:
            print(heap.delete())

if __name__ == '__main__':
    main()

```

代码运行截图（至少包含有"Accepted"）

#50795511提交状态

查看 提交 统计 提问

状态: Accepted

源代码

```
class BinaryHeap:
    def __init__(self):
        self.heap = []

    def insert(self, n):
        self.heap.append(n)
        temp_index = len(self.heap) - 1
        while (temp_index - 1)//2 >= 0:
            parent_index = (temp_index - 1)//2
            if self.heap[parent_index] > self.heap[temp_index]:
                self.heap[parent_index], self.heap[temp_index] = self.heap[temp_index], self.heap[parent_index]
                temp_index = parent_index
            else:
                break

    def delete(self):
        self.heap[0], self.heap[-1] = self.heap[-1], self.heap[0]
        q = self.heap.pop()
        if len(self.heap) > 1:
            temp_index_0 = 0
            while 2*temp_index_0 + 1 < len(self.heap):
                child_index = 2 * temp_index_0 + 1
```

基本信息

#: 50795511

题目: 04078

提交人: 22n2200011816(略约横溪)

内存: 4100kB

时间: 659ms

语言: Python3

提交时间: 2025-11-11 17:20:37

2. 学习总结和个人收获

这周作业除了n皇后考了回溯，中间三道题考察了利用树的性质建树、遍历，最后两道题则是树的应用。总体而言，中间三道题对我都是偏难，不过自己想出来之后确实有助于对树的理解；树的应用则是善用网络和讲义，也可以比较快地明白。